

W A S M B R I D G E

WEBASSEMBLY BRIDGE SYSTEM

ARCHITECTURE / API / USAGE / PROGRAMMING REFERENCE

SYSTEMS PROGRAMMER'S MANUAL

RELEASE 0.8.0

OCTOBER 2026

TARGET BASELINE

WINDOWS XP x64 / FIREFOX 52.9 ESR / wasm32

PLUS MODERN wasm32 AND memory64 ENGINES

MAGNETICON / WASMBRIDGE

BLACK-AND-WHITE TECHNICAL EDITION

00 PUBLICATION NOTE

BASELINE // This manual describes WasmBridge release 0.8.0 as implemented in the Magneticon/WasmBridge source tree and validated on the project's Windows XP / Firefox 52 target.

WasmBridge is a reusable WebAssembly compilation, packaging and browser-runtime framework. Its unusual compatibility floor is Firefox 52.9 ESR on Windows XP, while the same architecture also supports newer WebAssembly engines and the memory64 address model.

The 0.8 series removes the prototype-era fixed 16 MiB per-buffer, 32 MiB linear-memory and 128-live-allocation ceilings. Resource limits are now determined by the WebAssembly engine, browser/native process address space, allocator success, or optional quotas explicitly configured by the embedding application.

ACCEPTANCE STATUS AT PUBLICATION

TEST	RESULT
Windows XP-targeted native x86 build/self-test	PASS
Windows XP-targeted native x64 build/self-test	PASS
wasm32 direct binary tests and allocator hardening	PASS
memory64 / BigInt host regression	PASS on supporting engine
WABT 1.0.42 legacy validation	PASS
Binaryen 133 optimize + wasm2js generation	PASS
esbuild 0.28.2 Firefox 52 syntax gate	PASS
Windows XP x64 / Firefox 52 native-WASM acceptance	PASS

The XP browser acceptance run passed HelloWorld, Emscripten standalone Wasm, RGBA processing, BufferArena wasm32 with more than 128 live handles and more than 32 MiB of linear memory, and malformed-Wasm rejection.

NOTATION

MARK	MEANING
WASM	WebAssembly binary/module path.
JS	JavaScript host, adapter or fallback path.
wasm32	32-bit WebAssembly linear-memory addressing using i32 addresses.
memory64	64-bit WebAssembly linear-memory addressing using i64 / JavaScript BigInt.
HOST	Development or native Windows process.
XP	The Windows XP x64 legacy runtime/browser target.

01 CONTENTS

02	SYSTEM OVERVIEW
03	ARCHITECTURE
04	REPOSITORY AND TOOLCHAIN LAYOUT
05	BUILD AND COMPILATION MODEL
06	LOW-LEVEL LOADER API - WASMBRIDGE.LOAD
07	GENERAL MODULE API - WASMBRIDGEMODULE
08	BUFFERARENA AND RGBA ADAPTERS
09	MEMORY, ALLOCATION AND ADDRESS WIDTH
10	C ABI AND ALLOCATOR
11	JAVASCRIPT HOST AND FALLBACK PROGRAMMING
12	C AND C++ PROGRAMMING REFERENCE
13	C# MIGRATION GUIDANCE
14	PACKAGE FORMAT AND PACKAGE LOADER
15	COMMAND-LINE INTERFACE REFERENCE
16	FIREFOX 52 / WINDOWS XP COMPATIBILITY CONTRACT
17	TESTING AND VALIDATION
18	ERROR MODEL AND TROUBLESHOOTING
19	SECURITY AND DEPLOYMENT NOTES
20	QUICK REFERENCE TABLES
A	EXAMPLE END-TO-END MODULE
B	RELEASE CHECKLIST

02 SYSTEM OVERVIEW

WasmBridge separates portable computation from browser/operating-system services. JavaScript remains responsible for DOM, UI, networking, storage and browser integration. WebAssembly is used for compact, deterministic computational kernels and libraries that can be expressed behind a flat ABI.

DESIGN OBJECTIVES

- * Run native WebAssembly in Firefox 52.9 ESR on Windows XP x64.
- * Use the same runtime architecture on modern browsers and modern WebAssembly engines.
- * Support both wasm32 and memory64 address widths without conflating them with native process bitness.
- * Permit JavaScript fallbacks for engines where Wasm is unavailable, disabled or rejected.
- * Package a module, fallback, runtime, optional adapter and ABI metadata into a reproducible directory.
- * Keep the SDK self-contained by bundling pinned build-host dependencies under Toolchain/.
- * Avoid arbitrary framework memory quotas; application policy may add quotas explicitly.

WHAT WASMBRIDGE IS NOT

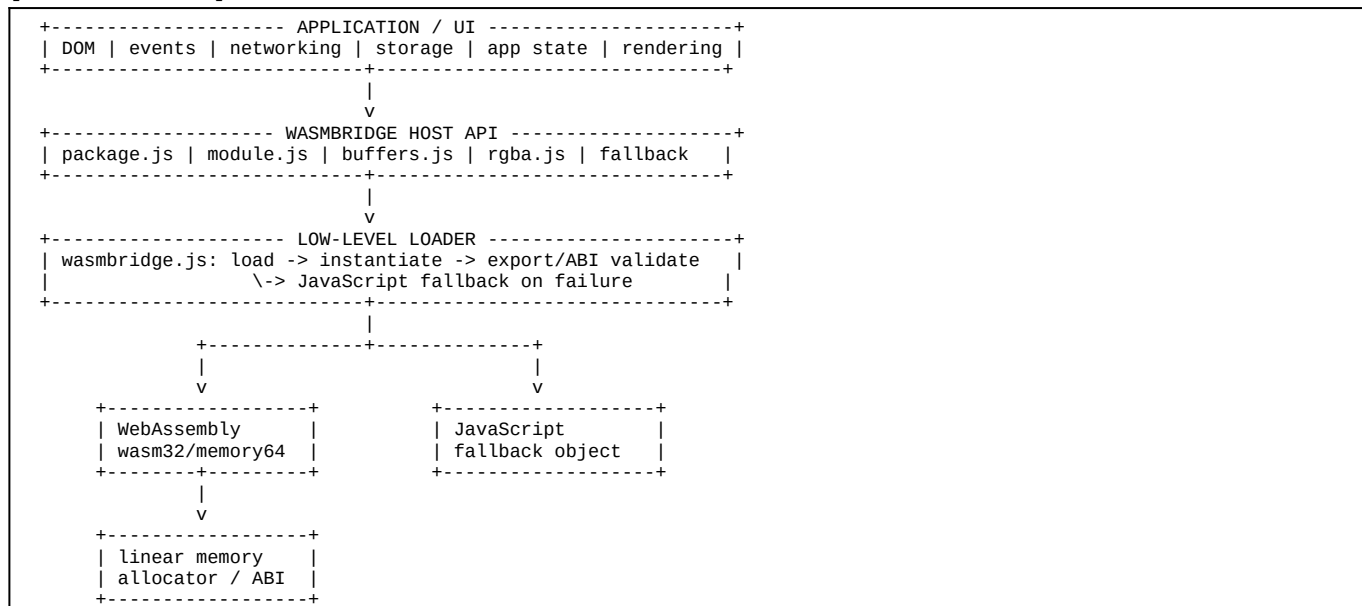
- * It is not a universal JavaScript-to-Wasm transpiler.
- * It is not a CLR or direct C# browser runtime.
- * It does not automatically provide DOM, Win32, COM, CUDA, filesystem or socket APIs inside a Wasm module.
- * It is not a general WebAssembly polyfill. A wasm2js candidate must still pass module-specific parity tests.
- * Manifest SHA-256 fields provide integrity/audit information; they are not publisher signatures.

RULE OF THUMB // Keep stateful/browser-specific orchestration in JavaScript. Move pure, buffer-oriented, CPU-intensive or binary algorithms into C/C++/Wasm.

03 ARCHITECTURE

LAYERED MODEL

[ARCHITECTURE MAP]



RUNTIME COMPONENTS

FILE	ROLE
Runtime/wasmbridge.js	Low-level wasm/fallback loader; export validation and failure phase reporting.
Runtime/module.js	General module abstraction; calls, signatures, owned handles, strings, typed copies, growth, disposal.
Runtime/buffers.js	Specialized BufferArena adapter and RGBA buffer processing helpers.
Runtime/rgba.js	Fixed scratch-buffer demonstration adapter with intentional 512x512 bound.
Runtime/package.js	Manifest-driven loader for v0.1/v0.2 packages and optional adapters.
Runtime/legacy-compat.js	XHR ArrayBuffer loader for environments where fetch is unavailable.
Core/buffers.c	Portable dynamic allocation ABI and RGBA processing kernel.

DATA-FLOW PRINCIPLE

The browser host owns JavaScript objects. The Wasm module owns linear-memory bytes. WasmBridge handles are ownership tokens that connect the two. A handle identifies a raw address, checked byte offset, capacity, logical length and address width. The general API returns copies instead of persistent typed-array views because `memory.grow()` may replace the backing `ArrayBuffer`.

04 REPOSITORY AND TOOLCHAIN LAYOUT

[REPOSITORY MAP]

```

WasmBridge\
  CLI\           .NET 4.0 command-line frontend
  Compiler\      compiler, package and external-tool wrappers
  Core\          portable C ABI and algorithms
  Native\        XP-targeted native DLL project
  Runtime\       browser runtime layers
  Examples\      browser acceptance fixtures
  Tests\         Node/PowerShell/native regressions
  Tools\         build, validation, release, server scripts
  Toolchain\     pinned self-contained host toolchain
  Documentation\ source documentation
  bin\Release\WXP\ x86/x64 XP-target binaries

```

BUNDLED TOOLCHAIN

COMPONENT	PINNED VERSION / PURPOSE
WABT	1.0.42 - legacy validation and WebAssembly inspection.
Binaryen	version_133 - wasm-opt optimization and wasm2js generation.
esbuild	0.28.2 - classic-script output and Firefox 52 syntax gate.
emsdk	Pinned Emscripten environment; also supplies upstream LLVM/Clang/wasm-ld.
Firefox52-TestProfile	Isolated browser test profile used by automated acceptance where available.

XP RUNTIME // The Windows XP browser machine does not need Node, LLVM, Emscripten, WABT, Binaryen, esbuild or Visual Studio. Those are development-host SDK components only.

05 BUILD AND COMPILATION MODEL

WINDOWS HOST BINARIES

- * Visual Studio 2022 solution: WasmBridge.sln.
- * Platform toolset: v141_xp.
- * Managed target: .NET Framework 4.0.
- * Native/managed configurations: x86 and x64.
- * Windows SDK baseline: 7.1A for XP-target builds.

[HOST BUILD]

```
powershell -ExecutionPolicy Bypass -File Tools\Build-Host.ps1 -Architecture x86
powershell -ExecutionPolicy Bypass -File Tools\Build-Host.ps1 -Architecture x64
```

DIRECT C TO WASM

[CLI]

```
WasmBridge.exe build --source Core\math.c --out add.wasm --export add
WasmBridge.exe build --source Core\buffers.c --out buffers.wasm --address-bits 32 --export
wb_alloc,wb_free,wb_capacity,wb_active_count,wb_invert_rgba,wb_address_bits
WasmBridge.exe build --source Core\buffers.c --out buffers64.wasm --address-bits 64 --export
wb_alloc,wb_free,wb_capacity,wb_active_count,wb_invert_rgba,wb_address_bits
```

The default is `--address-bits 32`. Direct compilation uses the bundled emsdk upstream Clang/wasm-ld when present, unless an explicit `--clang` path or environment override is supplied.

EMSCRIPTEN

[CLI]

```
WasmBridge.exe build-emscripTEN --source library.cpp --out library.wasm --export function1,function2
WasmBridge.exe build-emscripTEN --source library.cpp --out library64.wasm --address-bits 64 --export
function1,function2
```

The Emscripten backend emits standalone Wasm without Emscripten JS glue. For address-bits 64 it requests `MEMORY64=1`. Firefox 52 packages must remain `wasm32`.

LEGACY COMPATIBILITY GATE

`validate-legacy` is intentionally conservative. It uses WABT to reject post-MVP feature families that are inappropriate for Firefox 52. A `memory64` module is a modern-engine artifact and is expected to fail this legacy gate.

06 LOW-LEVEL LOADER API - WASMBRIDGE.LOAD

[SIGNATURE]

```
WasmBridge.load(options) -> Promise<adapter>
```

IMPORTANT OPTIONS

OPTION	MEANING
wasm	URL of the .wasm binary.
fallback	Fallback object, factory function, or script URL.
fallbackGlobal	Global name expected from fallback script; default WasmBridgeFallback.
imports	WebAssembly import object.
exports	Required callable export names; default ["add"].
preferWasm	Set false to bypass Wasm and use fallback.
validateWasm	Optional callback that validates instance exports/ABI after instantiation.

RESOLUTION SEQUENCE

[LOADER STATE MACHINE]

1. Validate loader configuration.
2. If preferWasm == false -> select fallback (phase = disabled).
3. Ensure WebAssembly.instantiate is available.
4. Load bytes using fetch, or WasmBridgeCompat.loadBytes via XHR.
5. Instantiate with options.imports.
6. Verify every required export is callable.
7. Run validateWasm(exports, instance), if supplied.
8. Return backend = "wasm" adapter.
9. On any Wasm failure, load/validate fallback and return backend = "javascript".

ADAPTER PROPERTIES

PROPERTY	MEANING
backend	"wasm" or "javascript".
diagnostic	Fallback diagnostic string; empty for successful Wasm.
failurePhase	Phase that caused Wasm fallback, otherwise null.
wasmFailure	{phase,message} metadata for Wasm failure.
instance	WebAssembly.Instance for Wasm backend; null for JS fallback.
exports	Raw selected backend export object.
declared export name	Direct wrapper method for each required export.

FAILURE PHASES

PHASE	INTERPRETATION
configuration	Missing/invalid loader configuration.
disabled	Wasm disabled by caller.
unavailable	WebAssembly API absent.
load	Binary could not be fetched/loaded.
instantiate	Engine rejected or failed instantiation.
exports	Required callable export missing.
abi	validateWasm rejected the instantiated module.
unknown	Unclassified error passed into fallback logic.

07 GENERAL MODULE API - WASMBRIDGEMODULE

[SIGNATURE]

```
WasmBridgeModule.load(options) -> Promise<module>
```

The general module API builds on `WasmBridge.load` and adds allocator-aware ownership, pointer-width handling, typed copies, UTF-8 helpers, optional ABI signatures, retryable disposal and statistics.

CONFIGURATION OPTIONS

OPTION	MEANING
exports	Public callable exports exposed by <code>call()</code> .
allocator	{allocate, release, capacity?} function names.
memoryExport	Exported WebAssembly.Memory name; default "memory".
signatures	Optional type contracts for public exports.
addressBits	32 or 64. Auto-detected from <code>wb_address_bits()</code> when available; otherwise 32.
growMemory	Set false to prevent host-side memory.grow/retry.
maxBufferBytes	Optional application quota for one allocation.
maxMemoryBytes	Optional application quota for linear memory.
maxHandles	Optional application quota for live owned handles.

METHODS

METHOD	PURPOSE
<code>call(name, ...args)</code>	Invoke a declared public export; enforce signature if configured.
<code>allocate(bytes)</code>	Allocate an owned buffer handle.
<code>release(handle)</code>	Release a live owned handle.
<code>write(handle, data, offset?)</code>	Copy <code>Uint8Array/Uint8ClampedArray</code> bytes into Wasm memory.
<code>read(handle, count?, offset?)</code>	Return a copied <code>Uint8Array</code> .
<code>writeTyped(handle, type, array, elementOffset?)</code>	Copy a supported typed array.
<code>readTyped(handle, type, count?, elementOffset?)</code>	Return a copied typed array.
<code>writeString(handle, text, offset?, nullTerminate?)</code>	UTF-8 encode into an owned buffer.
<code>readString(handle, maxBytes?, offset?)</code>	Decode UTF-8 up to NUL or limit.
<code>allocateString(text)</code>	Allocate exact UTF-8 bytes + terminator and write text.
<code>withBuffer(bytes, callback)</code>	Scoped allocate/callback/release helper.
<code>dispose()</code>	Release all live handles; retryable on backend release failure.
<code>statistics()</code>	Return backend, address width, memory/handle counts and configured quotas.

BUFFER HANDLE

[HANDLE SHAPE]

```
{
  pointer:    Number (wasm32) | BigInt (memory64),
  byteOffset: checked Number for ArrayBuffer indexing,
  capacity:   Number,
  byteLength: requested logical length,
  addressBits: 32 | 64,
  _wasmBridgeOwner: internal ownership token
}
```

OWNERSHIP // Handles belong to exactly one module instance. Foreign handles and released handles are rejected before raw allocator calls are attempted.

SUPPORTED SIGNATURE TYPES

TYPE	JAVASCRIPT REPRESENTATION	LEGACY FIREFOX 52
i32	Number integer	YES
i64	BigInt	NO
f32	Number	YES
f64	Number	YES
void	undefined result	YES

08 BUFFERARENA AND RGBA ADAPTERS

WASMBRIDGEBUFFERS

`WasmBridgeBuffers.load(options)` is a specialized adapter over the general allocator ABI. It provides allocation/copy helpers plus `invertRGBA/processRGBA` convenience functions. In 0.8 it no longer imposes the old 16 MiB per-buffer, 32 MiB total-memory or 128-handle framework limits.

WASMBRIDGERGBA

`WasmBridgeRGBA` is intentionally different: it demonstrates a single fixed scratch buffer exported by `Core/image.c`. Its 512x512 pixel bound is a property of that demonstration ABI, not a `WasmBridge`-wide memory limit.

USE THE RIGHT ADAPTER // For arbitrary-size or general-purpose binary data, use <code>WasmBridgeModule</code> or <code>WasmBridgeBuffers</code>. Use <code>WasmBridgeRGBA</code> only when the fixed scratch-buffer demo ABI is exactly what you want.
--

09 MEMORY, ALLOCATION AND ADDRESS WIDTH

TWO INDEPENDENT WIDTHS

[IMPORTANT DISTINCTION]

NATIVE PROCESS WIDTH	WEBASSEMBLY ADDRESS WIDTH
32-bit Firefox	wasm32 (i32 addresses)
64-bit Firefox	wasm32 OR memory64
Windows XP x64	does NOT imply memory64
Modern 64-bit process	does NOT imply memory64

WASM32

wasm32 pointers are 32-bit addresses. WebAssembly i32 results are surfaced to JavaScript as signed Numbers. An address above 0x7fffffff may therefore appear negative. WasmBridge canonicalizes such results into the unsigned 0..0xffffffff range before bounds checking or indexing linear memory.

The theoretical address range is 4 GiB. A 32-bit Firefox process can usually use substantially less because JavaScript heaps, JIT code, browser structures and other allocations share the native process virtual address space.

MEMORY64

memory64 modules use i64 linear-memory addresses. JavaScript sees these as BigInt. WasmBridge preserves the raw BigInt in handle.pointer and separately stores a checked Number byteOffset for current ArrayBuffer/typed-array indexing. Addresses outside JavaScript safe numeric indexing are rejected instead of rounded.

GROWTH POLICY

WasmBridge 0.8 has no default maximum buffer size, memory byte count or live-handle count. allocate() first asks the module allocator. If allocation fails and growMemory is not false, the runtime grows memory and retries. If memory.grow() or the allocator ultimately fails, allocation fails.

Optional maxBufferBytes, maxMemoryBytes and maxHandles are application sandboxing policy. They are not compatibility limits.

MEMORY.GROW AND VIEWS

A WebAssembly.Memory growth may replace/detach the backing ArrayBuffer. WasmBridge therefore creates fresh views for each operation and returns copies to callers. Do not cache typed-array views to memory.buffer across calls that may grow memory.

10 C ABI AND ALLOCATOR

[BUFFER C ABI]

```
unsigned char *wb_alloc(wb_size_t bytes);
int           wb_free(unsigned char *pointer);
wb_size_t     wb_capacity(unsigned char *pointer);
wb_size_t     wb_invert_rgba(unsigned char *source,
                             unsigned char *destination,
                             wb_size_t bytes);
uint32_t      wb_active_count(void);
uint32_t      wb_address_bits(void);
```

`wb_size_t` is 32-bit for `wasm32` and 64-bit for `wasm64`. `wb_address_bits()` reports the compiled pointer width so the JavaScript runtime does not have to guess.

ALLOCATOR BEHAVIOR

- * Wasm builds store allocation headers inside linear memory instead of a fixed descriptor table.
- * Native builds maintain dynamic metadata around normal native allocations.
- * Free blocks are split when reused for smaller requests.
- * Adjacent free blocks coalesce.
- * Free tail address space is reclaimed for later allocation, although `WebAssembly.Memory` itself never shrinks.
- * Unknown or double-freed raw pointers are rejected by `wb_free`.
- * The JavaScript layer additionally prevents stale or foreign handle use.

RECOMMENDED EXPORTED ABI STYLE

Use flat C functions. Prefer scalar numbers and pointer+length pairs. Keep C++ classes, STL containers, exceptions and implementation layout internal to the module.

11 JAVASCRIPT HOST AND FALLBACK PROGRAMMING

HOST RESPONSIBILITIES

- * DOM and UI operations.
- * Network requests and authentication.
- * Browser storage and application state.
- * Conversion between application objects and flat module buffers.
- * Module selection, fallback policy and error presentation.

FALLBACK CONTRACT

A fallback should expose the same callable export names as the Wasm path. If higher-level adapters rely on allocator/memory behavior, the fallback must also emulate that contract closely enough for the adapter to operate.

[FALLBACK OBJECT]

```
var MyFallback = {
  add: function (a, b) { return (a + b) | 0; },
  process: function (ptr, bytes) { /* equivalent behavior */ return 0; }
};

WasmBridge.load({
  wasm: "module.wasm",
  fallback: MyFallback,
  exports: ["add", "process"]
}).then(function (api) {
  console.log(api.backend); // "wasm" or "javascript"
});
```

TESTING PRINCIPLE // Keep the fallback independent enough to catch errors in the native implementation. Byte-for-byte parity tests are preferable to two implementations that share the same bug.

FIREFOX 52 JAVASCRIPT BASELINE

- * Classic scripts only.
- * No ES modules/import/export.
- * No async/await.
- * No optional chaining.
- * No BigInt literals in files loaded by Firefox 52.
- * Use ordinary functions, Promise chains and TypedArrays.

12 C AND C++ PROGRAMMING REFERENCE

SELECT THE WASM BOUNDARY

Move pure algorithms, codecs, parsers, image/audio transformations, math and binary processing into Wasm. Leave UI, files/networking, browser integration and platform APIs in JavaScript unless you explicitly provide imports.

C EXAMPLE

[C MODULE]

```
/* filter.c */
#include <stdint.h>

uint32_t invert_bytes(uint8_t *data, uint32_t bytes) {
    uint32_t i;
    if (!data) return 0;
    for (i = 0; i < bytes; ++i)
        data[i] = (uint8_t)(255u - data[i]);
    return bytes;
}
```

[BUILD]

```
WasmBridge.exe build \
--source filter.c \
--out filter.wasm \
--address-bits 32 \
--export invert_bytes
```

C++ WRAPPER EXAMPLE

[C++ ABI WRAPPER]

```
class FilterEngine {
public:
    static int apply(unsigned char *data, unsigned int bytes);
};

extern "C" int filter_apply(unsigned char *data, unsigned int bytes) {
    return FilterEngine::apply(data, bytes);
}
```

ABI DESIGN CHECKLIST

- * Export extern "C" wrappers from C++.
- * Use fixed-width integers where the semantic width matters.
- * Represent arrays/blobs as pointer + length.
- * Represent objects as integer handles/IDs, not exposed C++ object addresses/layout.
- * Return explicit status/error codes rather than allowing exceptions to cross the ABI.
- * For strings, use UTF-8 bytes and explicit ownership rules.
- * Avoid hidden ownership: document who allocates and who releases every cross-boundary buffer.
- * For Firefox 52, stay within wasm32 and MVP-oriented features.

CALLING FROM JAVASCRIPT

[HOST CALL]

```
WasmBridgeModule.load({
  wasm: "filter.wasm",
  fallback: FilterFallback,
  exports: ["invert_bytes"],
  allocator: {
    allocate: "wb_alloc",
    release: "wb_free",
    capacity: "wb_capacity"
  }
}).then(function (m) {
  var h = m.allocate(input.length);
  try {
    m.write(h, input);
    var n = m.call("invert_bytes", h.pointer, h.byteLength);
    var output = m.read(h, n);
  } finally {
    m.release(h);
  }
});
```

13 C# MIGRATION GUIDANCE

WasmBridge itself does not compile C# and does not ship a CLR in Firefox. For the legacy Firefox 52 target, the dependable strategy is to extract algorithmic cores from C# and port them to C or C++ while keeping browser/UI orchestration in JavaScript.

C# CONCEPT	WASM-FRIENDLY REPRESENTATION
object reference	integer handle/ID or explicitly serialized structure
byte[] / numeric array	pointer + element/byte length
string	UTF-8 bytes + terminator or explicit length
delegate/callback	numeric callback ID or Wasm import
exception	explicit status/error code
P/Invoke / Win32	host-side implementation; not directly available in browser Wasm
long / ulong	i64 / BigInt on modern engines; avoid in Firefox 52 browser contracts

A modern external C#-to-Wasm compiler may be used, but its output is simply a third-party Wasm module from WasmBridge's perspective. It must obey the target engine feature set, import requirements and WasmBridge ABI contract.

14 PACKAGE FORMAT AND PACKAGE LOADER

PACKAGE VERSIONS

FORMAT	PURPOSE
wasmbridge-package-0.1	Low-level package: Wasm, fallback, wasmbridge.js, exports, optional adapter.
wasmbridge-package-0.2	General-module package: adds module.js, allocator metadata, memory export and optional signatures.

TYPICAL V0.2 MANIFEST

[MANIFEST EXAMPLE]

```
{
  "format": "wasmbridge-package-0.2",
  "wasm": "module.wasm",
  "fallback": "fallback.js",
  "runtime": "wasmbridge.js",
  "moduleRuntime": "module.js",
  "adapter": "adapter.js",
  "exports": ["process", "active_count"],
  "allocator": {
    "allocate": "wb_alloc",
    "release": "wb_free",
    "capacity": "wb_capacity"
  },
  "memoryExport": "memory",
  "fallbackGlobal": "MyFallback",
  "signatures": {
    "process": {"parameters": ["i32", "i32"], "result": "i32"}
  },
  "wasmSha256": "...",
  "fallbackSha256": "...",
  "runtimeSha256": "..."
}
```

PACKAGE VERIFICATION

- * Artifact paths must be relative and must remain inside the package directory.
- * Required artifacts must exist.
- * SHA-256 values must match.
- * The Wasm binary must have the WebAssembly magic/version header.
- * Export names must be non-empty and unique.
- * Signatures may only refer to declared exports and supported ABI types.
- * Unexpected extra artifacts are rejected by verify-package.

SECURITY // Manifest hashes detect accidental or unauthorized modification relative to the manifest, but they do not authenticate the publisher. Serve packages only from trusted origins.

BROWSER PACKAGE LOADER

WasmBridgePackage.load() fetches/parses the manifest, loads wasmbridge.js if needed, loads an optional adapter script, loads module.js for v0.2 if needed, then constructs either the low-level or general module. The returned object is annotated with packageManifest, packageUrl and packageAdapterLoaded.

15 COMMAND-LINE INTERFACE REFERENCE

COMMAND	PURPOSE
build	Compile C directly with Clang/wasm-ld.
build-emscripten	Compile C/C++ with Emscripten into standalone Wasm.
validate-legacy	Validate Firefox-52/MVP-oriented wasm32 profile with WABT.
optimize-legacy	Optimize using Binaryen wasm-opt, then revalidate.
generate-fallback	Generate wasm2js candidate and esbuild it for Firefox 52.
package	Create v0.1/v0.2 package directory and manifest.
verify	Verify WebAssembly binary magic/version header.
verify-package	Verify package paths, hashes, declared artifacts and header.
self-test	Run managed header checks and native ABI self-tests.

SYNTAX SUMMARY

[CLI QUICK REFERENCE]

```

build --source FILE --out FILE [--clang FILE]
    [--export A,B] [--address-bits 32|64] [--timeout MS]

build-emscripten --source FILE --out FILE [--emcc FILE]
    [--export A,B] [--address-bits 32|64]
    [--include A;B] [--define A;B] [--timeout MS]

validate-legacy --wasm FILE [--validator FILE]
optimize-legacy --wasm FILE --out FILE [--optimizer FILE] [--validator FILE]
generate-fallback --wasm FILE --out FILE [--validator FILE]
    [--wasm2js FILE] [--esbuild FILE] [--global NAME]

package --wasm FILE --fallback FILE --runtime FILE --out DIR
    [--adapter FILE] [--module-runtime FILE] [--export A,B]
    [--allocator alloc,free[,capacity]] [--memory-export NAME]
    [--fallback-global NAME] [--signatures FILE]

verify --wasm FILE
verify-package --manifest FILE
self-test

```

PARSER BEHAVIOR // Unknown switches are rejected. This prevents misspelled options from silently falling back to defaults.

16 FIREFOX 52 / WINDOWS XP COMPATIBILITY CONTRACT

Actual execution on Firefox 52.9 ESR is the acceptance criterion. Host-side syntax gates and WABT validation are preflight checks, not substitutes for the legacy browser.

BROWSER BASELINE

- * Firefox 52 uses wasm32 even on Windows XP x64.
- * Classic JavaScript scripts only.
- * No ES module syntax.
- * No async/await or optional chaining.
- * No BigInt literals in legacy-loaded source files.
- * Runtime memory64 branches must remain dormant on Firefox 52.
- * Use XHR compatibility loader when fetch is unavailable.

DEFINITIVE LAN ACCEPTANCE ROUTE

[ACCEPTANCE]

```
node Tools\Serve-Examples.js 8084 192.168.255.2
```

```
XP Firefox 52:  
http://192.168.255.2:8084/Examples/WasmSelfTest/index.html
```

The XP machine needs only Firefox. The server/toolchain remain on the Windows 10 development host.

VALIDATED XP SELF-TEST ITEMS

ITEM	STATUS
HelloWorld native Wasm	PASS
Emscripten standalone Wasm	PASS
RGBA native Wasm	PASS
BufferArena wasm32, >128 handles, >32 MiB	PASS
Malformed Wasm rejection	PASS

17 TESTING AND VALIDATION

THREE LAYERS

1. Direct binary tests instantiate shipped .wasm binaries without the WasmBridge wrapper or JS fallback.
2. Runtime/adaptor regressions test loaders, handles, memory growth, fallbacks, package behavior and error phases.
3. Real Firefox 52 acceptance tests execute in the actual target browser on Windows XP.

COMPLETE HOST MATRIX

[WINDOWS 10 HOST]

<code>powershell -ExecutionPolicy Bypass -File Tools\Test-All.ps1 -SkipFirefox</code>

The host matrix builds XP-targeted x86/x64 binaries, runs native/managed self-tests, package integrity tests, Emscripten contract tests, Node runtime tests, WABT validation, Binaryen optimization/wasm2js generation, fresh fallback parity and esbuild Firefox 52 syntax checks.

IMPORTANT 0.8 REGRESSIONS

- * A single allocation larger than the old 16 MiB cap.
- * Linear-memory growth beyond the old 32 MiB cap.
- * More than 128 simultaneous live handles.
- * Free-block splitting, reuse and coalescing.
- * 2048x2048 RGBA BufferArena processing.
- * Retryable dispose() after backend release failure.
- * Cross-realm typed arrays.
- * Package adaptor loading.
- * Synchronous instantiate failure phase retention.
- * memory64 i64/BigInt behavior on supporting engines.

18 ERROR MODEL AND TROUBLESHOOTING

SYMPTOM	LIKELY CAUSE / ACTION
Wasm falls back with [load]	URL/HTTP/fetch/XHR failure. Verify server path and same-origin access.
Wasm falls back with [instantiate]	Unsupported feature, malformed binary, missing imports, engine rejection.
Wasm falls back with [exports]	Required export absent or non-callable.
Wasm falls back with [abi]	validateWasm rejected memory or contract.
Allocation fails after growth	Engine/address space/allocator refused request or configured quota reached.
Handle rejected as foreign	Handle came from another module instance.
Handle rejected as released	Handle was already released; allocate a new buffer.
memory64 load fails on Firefox 52	Expected: Firefox 52 supports wasm32 only.
validate-legacy rejects module	Module uses features outside the restricted legacy profile.
Fallback candidate mismatch	Generated wasm2js semantics differ from maintained fallback/ABI; investigate before shipping.

DISPOSAL FAILURES

dispose() is intentionally retryable. If one backend release fails, handles successfully released are removed, failed handles remain active, and the module is not marked disposed. Correct the backend condition if possible and call dispose() again.

19 SECURITY AND DEPLOYMENT NOTES

- * Run only trusted Wasm binaries and JavaScript fallbacks.
- * Fallback scripts execute with the privileges of the embedding origin.
- * Package hashes do not replace code signing or trusted transport.
- * Validate imported functions and exposed ABI contracts; do not assume syntactic validity implies safe semantics.
- * Prefer HTTP(S) serving for reproducible browser behavior rather than relying on file:// quirks.
- * Treat optional application quotas as a defense-in-depth policy where untrusted workloads may request memory.
- * Keep build-host toolchain dependencies off the XP runtime machine unless they are explicitly needed for development.

20 QUICK REFERENCE TABLES

RUNTIME GLOBAL OBJECTS

GLOBAL	VERSION	PURPOSE
WasmBridge	0.8.0	Low-level loader/fallback selector.
WasmBridgeModule	0.8.x	General allocator-aware module API.
WasmBridgeBuffers	0.8.x	BufferArena specialized adapter.
WasmBridgeRGBA	0.8.x	Fixed scratch-buffer demonstration adapter.
WasmBridgePackage	0.8.0	Manifest-driven package loader.
WasmBridgeCompat	legacy helper	XHR byte loading when fetch is unavailable.

TYPED-COPY NAMES

NAME	TYPEDARRAY
u8	Uint8Array
u8c	Uint8ClampedArray
i8	Int8Array
u16	Uint16Array
i16	Int16Array
u32	Uint32Array
i32	Int32Array
f32	Float32Array
f64	Float64Array

ADDRESSING SUMMARY

PROPERTY	WASM32	MEMORY64
Raw pointer type	i32	i64
JavaScript pointer	Number (canonicalized unsigned)	BigInt
Firefox 52	YES	NO
Automatic detection	wb_address_bits() -> 32	wb_address_bits() -> 64
Manual override	addressBits: 32	addressBits: 64

A EXAMPLE END-TO-END MODULE

1. C KERNEL

[SOURCE]

```
#include "buffers.h"

uint32_t threshold_rgba(unsigned char *p, uint32_t bytes,
                        unsigned char cutoff) {
    uint32_t i;
    if (!p || (bytes & 3u)) return 0;
    for (i = 0; i < bytes; i += 4) {
        unsigned int y = (unsigned int)p[i] + p[i+1] + p[i+2];
        unsigned char v = (y / 3u) >= cutoff ? 255 : 0;
        p[i] = p[i+1] = p[i+2] = v;
        /* alpha p[i+3] preserved */
    }
    return bytes;
}
```

2. BUILD

[BUILD]

```
WasmBridge.exe build --source threshold.c --out threshold.wasm --address-bits 32 --export
threshold_rgba,wb_alloc,wb_free,wb_capacity,wb_address_bits
```

3. HOST

[JAVASCRIPT]

```
WasmBridgeModule.load({
    wasm: "threshold.wasm",
    fallback: ThresholdFallback,
    exports: ["threshold_rgba"],
    allocator: {
        allocate: "wb_alloc",
        release: "wb_free",
        capacity: "wb_capacity"
    }
}).then(function (m) {
    var h = m.allocate(rgba.length);
    try {
        m.write(h, rgba);
        m.call("threshold_rgba", h.pointer, h.byteLength, 128);
        rgba = m.read(h);
    } finally {
        m.release(h);
    }
});
```

4. PACKAGE

[PACKAGE]

```
WasmBridge.exe package --wasm threshold.wasm --fallback threshold-fallback.js --runtime Runtime\wasmbridge.js --
module-runtime Runtime\module.js --out dist\threshold --export threshold_rgba --allocator
wb_alloc,wb_free,wb_capacity --memory-export memory --fallback-global ThresholdFallback
```

5. VALIDATE

[VALIDATION]

```
WasmBridge.exe verify-package --manifest dist\threshold\manifest.json
WasmBridge.exe validate-legacy --wasm dist\threshold\module.wasm
```

B RELEASE CHECKLIST

1. Build Release|x86 and Release|x64 using v141_xp / .NET Framework 4.0.
2. Run WasmBridge.exe self-test for both architectures.
3. Run package creation and tamper-detection tests.
4. Run Emscripten backend contract tests.
5. Run all Node runtime/direct-binary regressions.
6. Run WABT/Binaryen/esbuild external-toolchain validation from the bundled Toolchain/.
7. Run the Firefox 52 syntax gate.
8. Run the real XP Firefox 52 native-WASM acceptance page.
9. Verify release packages and SHA-256 sidecar.
10. Record tested versions and source commit in release notes.

PUBLICATION SOURCES

This manual consolidates the release 0.8 README, API reference, programmer's guide, memory/addressing notes, Firefox 52 compatibility contract, and the current implementations of Runtime/wasmbridge.js, Runtime/module.js, Runtime/package.js and Compiler/WasmPackage.cs. Where prose and implementation differ, the current implementation is authoritative.

END OF SYSTEMS PROGRAMMER REFERENCE

WASMBRIDGE 0.8.0 // MAGNETICON

WINDOWS XP COMPATIBILITY PRESERVED